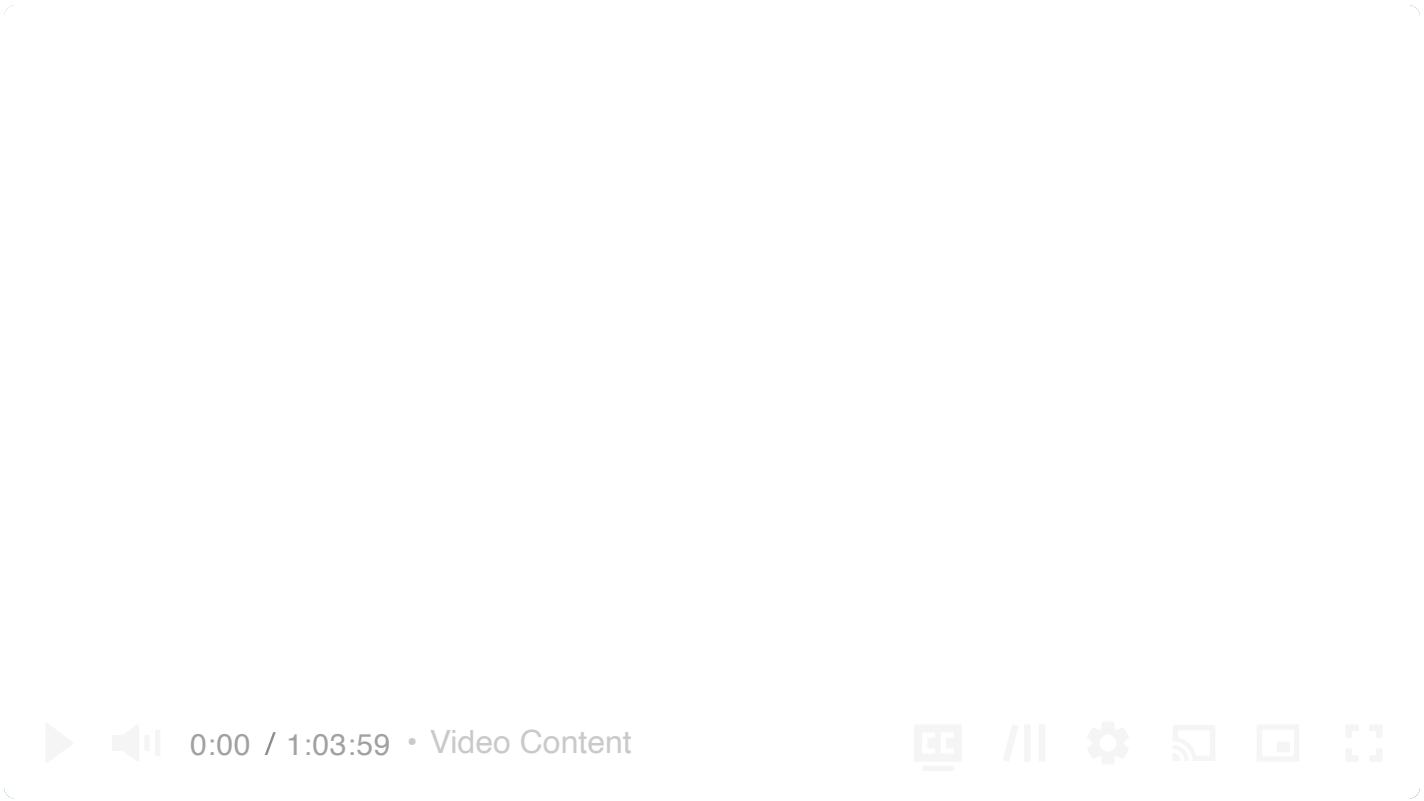
Common Problems

Payment System

Multi-step Processes

By Evan King · Updated Jul 11, 2025 · hard · Asked at: A a C +1



Understanding the Problem

💣 **What is Stripe?**

Payment processing systems like Stripe allow business (referred to throughout this breakdown as merchants) to accept payment from customers, without having to build their own payment processing infrastructure. Customer input their payment details on the merchant's website, and the merchant sends the payment details to Stripe. Stripe then processes the payment and returns the result to the merchant.

Functional Requirements

Core Requirements

1. Merchants should be able to initiate payment requests (charge a customer for a specific amount).
2. Users should be able to pay for products with credit/debit cards.
3. Merchants should be able to view status updates for payments (e.g., pending, success, failed).

Below the line (out of scope):

- Customers should be able to save payment methods for future use.
- Merchants should be able to issue full or partial refunds.
- Merchants should be able to view transaction history and reports.
- Support for alternative payment methods (e.g., bank transfers, digital wallets).
- Handling recurring payments (subscriptions).
- Payouts to merchants.

Non-Functional Requirements

> Before defining your non-functional requirements in an interview, it's wise to inquire about the scale of the system as this will have a meaningful impact on your design. In this case, we'll be looking at a system handling about 10,000 transactions per second (TPS) at peak load.

Core Requirements

1. The system should be highly secure
2. The system should guarantee durability and auditability with no transaction data ever being lost, even in case of failures.
3. The system should guarantee transaction safety and financial integrity despite the inherently asynchronous nature of external payment networks
4. The system should be scalable to handle high transaction volume (10,000+ TPS) and potentially bursty traffic patterns (e.g., holiday sales).

Below the line (out of scope):

- The system should adhere to various financial regulations globally (depending on supported regions).
- The system should be extensible to easily add new payment methods or features later.

Here's how it might look on your whiteboard:

```
Functional Requirements
- Merchants should be able to create payments
- Users should be able to pay via credit/debit card
- Merchants should be able to view status updates for payments

Non-functional Requirements
- The system should be highly secure
- Transaction safety and financial integrity (no double charge, etc)
- Strong durability (can't lose transactions)
- Scale to 10k TPS
```

Payment System Requirements

The Set Up

Defining the Core Entities

Let's start by identifying the core entities we'll need. I prefer to begin with a high-level overview before diving into specifics as this helps establish the foundation we'll build upon. That said, if you're someone who finds value in outlining the complete schema upfront with all columns and relationships defined, that's perfectly fine too! There's no single "right" approach in an interview, just do what works best for you. The key is to have a clear understanding of the main building blocks we'll need.

To satisfy our key functional requirements, we'll need the following entities:

1. **Merchant:** This entity will store information about the businesses using our payment platform, including their identity details, bank account information, and API keys.
2. **PaymentIntent:** This represents the merchant's intention to collect a specific amount from a customer and tracks the overall payment lifecycle from creation to completion. It owns the state machine from created → authorized → captured / canceled / refunded, and enforces idempotency for retries.

3. **Transaction:** This represents a polymorphic money-movement record linked back to one PaymentIntent. Types include Charge (funds in), Refund (funds out), Dispute (potential reversal), and Payout (merchant withdrawal). Each row carries amount, currency, status, timestamps, and references to both the intent and the merchant. For simplicity, we'll only be focused on Charges as everything else is out of scope.

It's important to clarify the distinction between PaymentIntent and Transaction at this point as this can easily cause confusion. The relationship between them is one-to-many: a single PaymentIntent can have multiple Transactions associated with it. For example:

- If a payment attempt fails due to insufficient funds, the merchant might retry with the same PaymentIntent ID but a different Transaction will be created.
- For partial payments, multiple Transactions might be linked to the same PaymentIntent.
- For refunds, a new Transaction with a negative amount would be created and linked to the original PaymentIntent.

From the merchant's perspective, they simply create a PaymentIntent and our system handles all the transaction complexities internally, providing a simplified view of the overall payment status.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.

> A **Transaction** is a bit of a simplification to ensure we can focus on the important parts of the design with our limited time. In production you'd likely break this out into discrete types— `Charge`, `Refund`, `Dispute`, `Payout`, and the double-entry `LedgerEntry` rows that actually move balances. For interview purposes, though, collapsing them under a single polymorphic `Transaction` keeps the mental model tight: one `PaymentIntent` can spawn many money-movement events, each stamped with an amount, currency, status, and timestamps. That level of detail is enough to reason about idempotency, auditability, and failure handling without drowning in implementation minutiae.

```
Core Entities
- Merchant
- PaymentIntent
- Transaction
```

Payment Entities

API or System Interface

The API is the main way merchants will interact with our payment system. Defining it early helps us structure the rest of our design. We'll start simple and, as always, we can add more detail as we go. I'll just create one endpoint for each of our core requirements.

First, merchants need to initiate PaymentIntent requests. This will happen when a customer reaches the checkout page of a merchant's website. We'll use a `POST` request to create the PaymentIntent.

```
POST /payment-intents -> paymentIntentId
{
  "amountInCents": 2499,
  "currency": "usd",
  "description": "Order #1234",
}
```

Next, the system needs to securely accept and process payments. Since we're focusing on credit/debit cards initially, we'll need an endpoint for that:

```
POST /payment-intents/{paymentIntentId}/transactions
{
  "type": "charge",
  "card": {
    "number": "4242424242424242",
    "exp_month": 12,
    "exp_year": 2025,
    "cvc": "123"
  }
}
```

> In a real implementation, we'd never pass raw card details directly to our backend like this. We'd use a secure tokenization process to protect sensitive data. We'll get into the details of how we handle this securely when we get further into our design. In your interview, I would just callout that you understand this data will need to be encrypted.

Finally, merchants need to check the status of payments. This can be done with a simple `GET` request like so:

```
GET /payment-intents/{paymentIntentId} -> PaymentIntent
```

The response would include the payment's current status (e.g., "pending", "succeeded", "failed") and any relevant details like error messages for failed payments.

For a more real-time approach (and one that actually mirrors the industry standard), we could also provide webhooks that notify merchants when payment statuses change. In this way, the merchant would provide us with a callback URL that we would POST to when the payment status changes, allowing them to get real-time updates on the status of their payments.

```
POST {merchant_webhook_url}
{
  "type": "payment.succeeded",
  "data": {
    "paymentId": "pay_123",
    "amountInCents": 2499,
    "currency": "usd",
    "status": "succeeded"
  }
}
```

> Designing a webhook callback system is often a standalone question in system design interviews. Designing a payment system, complete with webhooks, would be a lot to complete in the time allotted. Have a discussing with your interviewer early on so you're on the same page about what you're building.

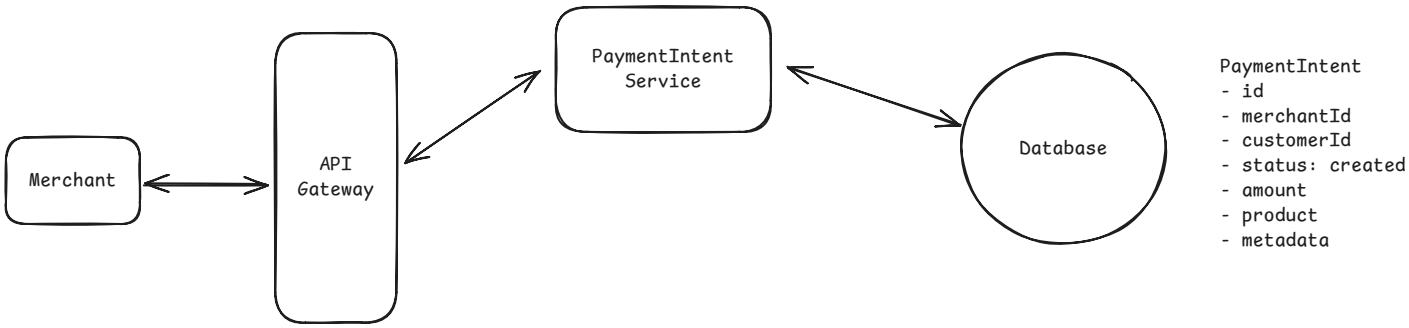
High-Level Design

For our high-level design, we're simply going to work one-by-one through our functional requirements.

1) Merchants should be able to initiate payment requests

When a merchant wants to charge a customer, they need to initiate a payment request. In our system, this is accomplished through the creation of a PaymentIntent. As we stated in our Core Entities, a PaymentIntent represents the merchant's intention to collect a specific amount from a customer and tracks the lifecycle of the payment from initiation to completion.

Let's start by laying out the core components needed for this functionality:



Payment Intent Flow

1. **API Gateway**: This serves as the entry point for all merchant requests. It handles authentication, rate limiting, and routes requests to the appropriate microservices.
2. **Payment Service**: This microservice is responsible for creating and managing PaymentIntents. It interfaces with our database to store payment information.
3. **Database**: A central database that stores all system data including PaymentIntents records (with their current status, amount, currency, and associated metadata) and merchant information (API keys, business details, and configuration preferences).

Here's the flow when a merchant initiates a payment:

1. The merchant makes an API call to our system by sending a POST request to `/payment-intents` with details like amount, currency, and description.
2. Once authenticated, the request is routed to the PaymentIntent Service (more on this later).

3. The PaymentIntent Service creates a new PaymentIntent record with an initial status of "created" and stores it in the Database.
4. The system generates a unique identifier for this PaymentIntent and returns it to the merchant in the API response.

This PaymentIntent ID is crucial as it will be used in subsequent steps of the payment flow. The merchant will typically embed this ID in their checkout page or pass it to their client-side code where it will be used when collecting the customer's payment details.

At this stage, no actual charging has occurred. We've simply recorded the merchant's intention to collect a payment and created a reference that will be used to track this payment throughout its lifecycle. The PaymentIntent exists in a "created" state, awaiting the next step where payment details will be provided and processing will begin.

2) Users should be able to pay for products with credit/debit cards.

Now that we have a PaymentIntent created, the next step is to securely collect payment details from the customer and process the payment. It's important to understand that payment processing is inherently asynchronous - the initial authorization response from the payment network is just the first step in a longer process that can take minutes or even days to fully complete. This is because payment networks need time to handle things like fraud checks, chargeback requests, etc.

Pattern: Multi-step Processes

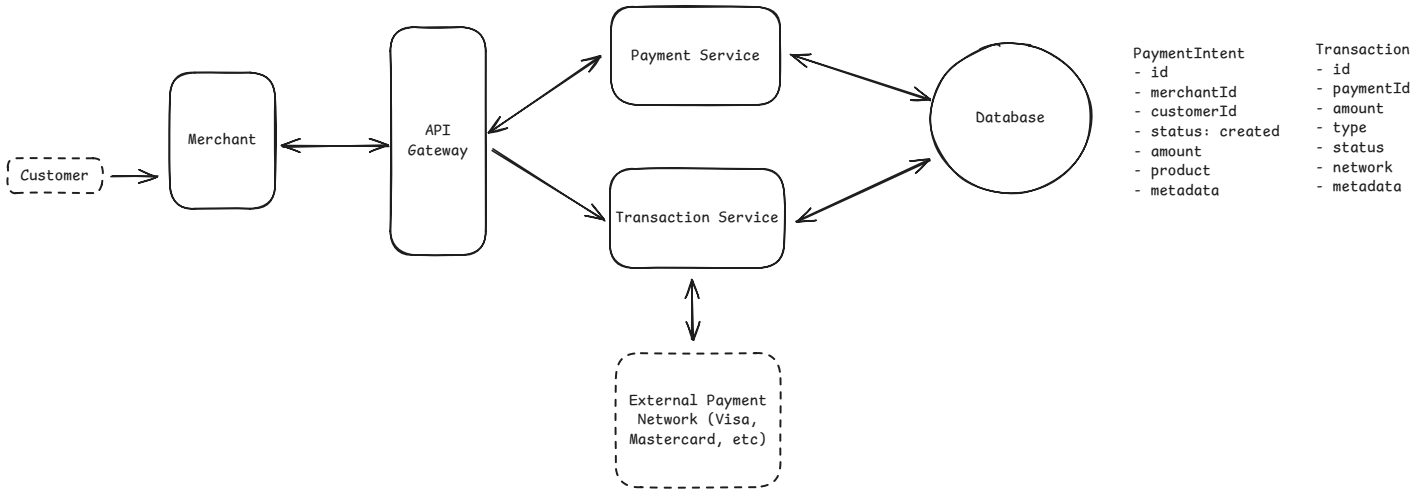
Payment processing is a perfect example of the multi-step processes pattern. A single payment goes through multiple stages: authorization, capture, settlement, and potentially refunds or disputes. Each step can fail independently and may require retries or compensation. This requires careful state management and orchestration to ensure the payment workflow completes successfully while handling failures gracefully.

Learn This Pattern

Our system needs to handle this asynchronous nature by maintaining the state of each payment and transaction, and keeping merchants informed of status changes throughout the

entire lifecycle.

Let's expand our architecture to handle this critical part of the payment flow:



Process Payment Flow

1. **Transaction Service**: A dedicated microservice responsible for receiving card details from the merchant server, managing transaction records throughout the payment lifecycle, and interfacing directly with external payment networks like Visa, Mastercard, and banking systems.
2. **External Payment Network**: Shown as a dotted line in our diagram to demonstrate that this is external to our actual system, though a crucial part of the payment flow. These are the payment networks (Visa, Mastercard, etc.) and banking systems that actually authorize and process the financial transactions.

Here's how the flow works when a customer enters their payment details:

1. The customer enters their credit card information into a payment form on the merchant's website.
2. The merchant collects this data and sends it to our Transaction Service along with the original PaymentIntent ID.
3. The Transaction Service creates a transaction record with status "pending" in our system.
4. The Transaction Service directly handles the payment network interaction: a. Connects to the appropriate payment network and sends the authorization request b. Receives the initial response (approval/decline) c. Updates the transaction record with the initial status d. Continues to listen for callbacks from the payment network over the secure private connection e. When additional status changes occur (settlement, chargeback, etc.), receives callbacks and updates records accordingly

5. The Transaction Service updates the PaymentIntent status as the transaction progresses through its lifecycle.

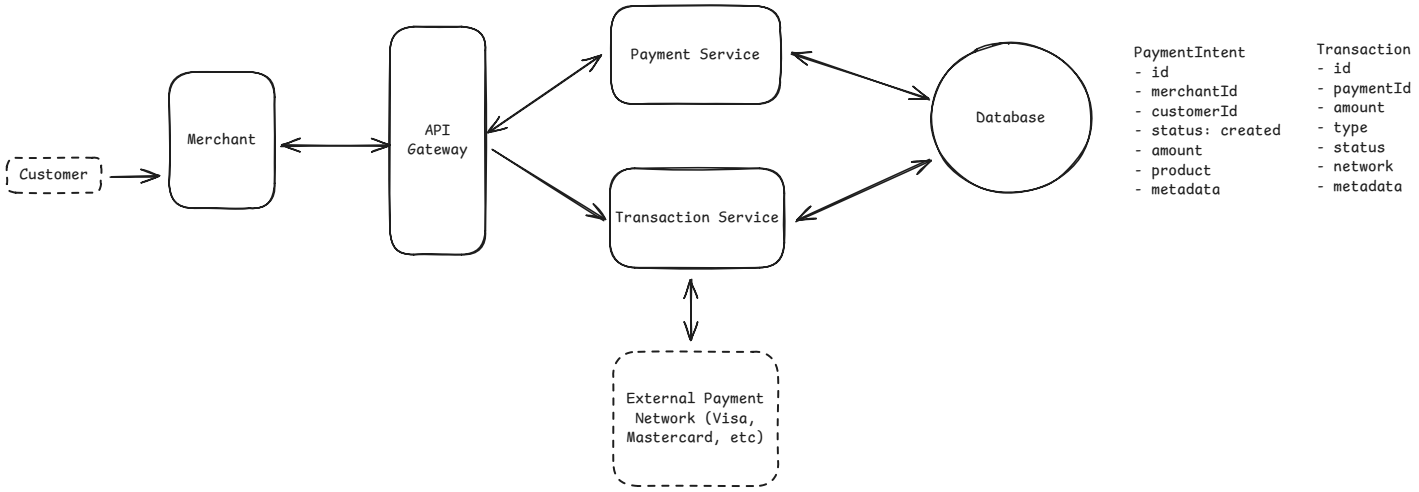
> While it would be highly unusual for any of these details to be covered in anything but the most specialized interviews, you may be wondering how connections to payment networks work.
>
> In short, payment networks operate on private, highly secure networks that are completely separate from the public internet. To connect with these networks, payment processors must establish dedicated connections through specialized hardware security modules (HSMs) and secure data centers that meet stringent Payment Card Industry Data Security Standard (PCI DSS) requirements. These private networks use proprietary protocols with specific message formats (such as ISO 8583) and require formal certification processes to gain access. Unlike typical REST APIs, these connections often involve binary protocols, leased lines, and VPN tunnels with mutual TLS authentication.

This simplified architecture keeps the Transaction Service as the single point of responsibility for both managing our internal transaction records and interfacing with external payment networks. While this combines multiple concerns in one service, it reduces complexity and eliminates unnecessary network hops while still maintaining security through proper PCI compliance within the service.

3) The system should provide status updates for payments

After a payment is initiated and processed, merchants need a reliable way to determine its current status. This information is very important for business operations! Merchants need to know when a payment succeeds to trigger fulfillment actions like shipping physical products, granting access to digital content, or confirming reservations. Likewise, they need to know when payments fail so they can notify customers or attempt alternative payment methods.

Let's see how our existing architecture supports this functionality:



Payment Status Flow

Since we already have a PaymentIntent Service that manages PaymentIntents, we can leverage this same service to provide status updates to merchants. There's no need to create a separate service just for checking statuses.

Here's how the flow works when a merchant checks a payment's status:

1. The merchant makes a GET request to `/payment-intents/{paymentIntentId}` to retrieve the current status of a specific PaymentIntent.
2. The API Gateway validates the merchant's authenticity and routes the request to the PaymentIntent Service.
3. The PaymentIntent Service queries the database for the current state of the PaymentIntent, including its status, any error messages (if failed), and related transaction details.
4. The service returns this information to the merchant in a structured response format.

This simple polling mechanism allows merchants to programmatically check payment statuses and integrate the results into their business workflows.

The PaymentIntent can have various statuses throughout its lifecycle, such as:

- `created` : Initial state after the merchant creates the PaymentIntent
- `processing` : PaymentIntent details received and being processed
- `succeeded` : PaymentIntent successfully processed
- `failed` : Payment processing failed (with reason)

While this polling approach works well for many use cases, it's not ideal for real-time updates or high-frequency status checks. In a deep dive later, we'll explore how webhooks can be implemented to provide push-based notifications that eliminate the need for polling and reduce latency between payment completion and fulfillment actions.

Now that we've covered all three functional requirements, we have a basic payment processing system that can create payments, process payments securely, and provide status updates to merchants.

Potential Deep Dives

At this point, we have a basic system that satisfies the core functional requirements of our payment processing system. Merchants can initiate payments by creating a PaymentIntent, customers can pay with credit/debit cards, and merchants can view payment status updates. However, our current design has significant limitations, particularly around transaction safety, durability, and scaling to handle high volumes. Let's look back at our non-functional requirements and explore how we can improve our system to handle 10,000+ TPS with strong consistency and guaranteed durability.

1) The system should be highly secure

Let's start with security. For a payment processing system, there are two main things we care about when it comes to guaranteeing the security of the system.

1. Is the person/merchant making the payment request who they say they are?
2. Are we protecting customer personal information so that it can't be stolen or compromised?

Starting with #1, we need to validate that merchants connecting to our system are who they claim to be. After all, we're giving them the ability to charge people money, we better make sure they're legit! Most payment systems solve this with API keys, but there are different approaches with varying levels of security. Here are some options.

Good Solution: Basic API Key Authentication

Great Solution: Enhanced API Key Management with Request Signing

Now let's look at #2 - protecting sensitive customer data throughout the payment process. Allowing a bad actor to get a hold of someone else's credit card information can lead to fraud, identity theft, and a total loss of customer trust. Plus, there are strict regulations like PCI DSS that mandate how payment data must be handled. Let's look at the different approaches to tackling this challenge.

Bad Solution: Server-Side Payment Data Collection

Good Solution: Client-Side iFrame Isolation

Great Solution: iFrame + Encryption

2) The system should guarantee durability and auditability with no transaction data ever being lost, even in case of failures.

For a payment system, the worst thing you can do is lose transaction data. It would be both a financial and legal disaster. Every transaction represents real money moving between accounts, and regulations like PCI-DSS, SOX compliance, and financial auditing standards require us to maintain complete, immutable records of every payment attempt, success, and failure.

We need to track not just what the current state is, but the entire sequence of events that led to that state. When a customer disputes a charge six months later, we must be able to prove exactly what happened: when the payment was initiated, what amount was authorized, when it was captured, and whether any refunds were processed. A single missing record could mean inability to defend against chargebacks, failed compliance audits, or worse—being unable to determine the true state of customer accounts.

This durability requirement intersects with several other system needs. As we'll explore later, webhook delivery requires knowing which events have been sent to which merchants. Reconciliation with payment networks demands comparing our records against theirs. Fraud

detection needs to analyze patterns across transaction history. All of these capabilities depend on having a durable, queryable audit trail as their foundation.

Let's examine different approaches to achieving this durability and auditability:

Bad Solution: Single Database with Current State Only

Good Solution: Separate Audit Tables

Great Solution: Database + Change Data Capture + Event Stream

By implementing this hybrid approach, we achieve true durability and auditability without sacrificing the performance our merchants expect. The immutable event stream becomes the foundation for not just audit compliance, but for the advanced capabilities—reconciliation, analytics, and real-time notifications—that distinguish a professional payment system from a simple transaction processor.

A savvy interviewer might ask: "Isn't CDC a single point of failure? What happens if your CDC system fails and you miss critical payment events?"

This is a great question! CDC is technically a single point of failure - if it stops working, events stop flowing to Kafka even though database writes continue. Companies like Stripe handle this by running multiple independent CDC instances reading from the same database, each writing to different Kafka clusters. They also implement monitoring that alerts within seconds if CDC lag increases, and maintain recovery procedures to replay missed events from database logs if needed. For critical payment events, they might also implement application-level fallbacks that write directly to Kafka if CDC hasn't confirmed the event within a certain timeframe.

3) The system should guarantee transaction safety and financial integrity despite the inherently asynchronous nature of external payment networks

Payment networks operate in a fundamentally different way than our internal systems. When we send a charge request to Visa, Mastercard, or a bank, we're crossing into systems we don't control. These networks process millions of transactions across global infrastructure, with their own retry mechanisms, queue delays, and batch processing windows. A payment we consider "timed out" might still be winding its way through authorization systems, while another might have succeeded instantly but lost its response packet on the way back to us.

This asynchronous reality creates serious risks. The most dangerous is double-charging. A customer clicks "pay" for their $50 purchase, we timeout waiting for the bank's response, the merchant retries, and suddenly the customer sees two $50 charges on their statement. Even if we eventually refund one, we've damaged trust and created support headaches. The opposite risk is equally damaging: a payment succeeds at the bank but we never know it, so we tell the merchant the payment failed. The merchant never ships the product, the customer's card gets charged anyway, and now we have an angry customer who paid for goods they'll never receive.

The challenge isn't eliminating this asynchronous behavior, which is impossible. Instead, we need to build systems that acknowledge uncertainty and handle it gracefully. Fortunately, the CDC and event stream infrastructure we established for durability provides exactly the foundation we need to track payment attempts, handle timeouts intelligently, and maintain consistency even when networks misbehave.

Let's explore different approaches to handling this inherent uncertainty:

Bad Solution: Assume Timeouts Mean Failure

Good Solution: Pending States with Manual Reconciliation

Great Solution: Event-Driven Safety with Reconciliation

In production payment systems like Stripe, transaction processing actually uses a two-phase event model:

1. **Transaction Created Event**: Emitted when the transaction service begins processing, before the database write is complete. If this event fails to emit, the transaction enters a locked/failed state and can be retried.

> 2. **Transaction Completed Event**: Emitted after the database write has successfully completed. If this event fails to emit, the transaction also enters a locked/failed state where further updates are blocked until the completion event is successfully
>
> Show More

The key to handling asynchronous payment networks is accepting that uncertainty is inevitable and building systems designed for eventual consistency. By combining a traditional database for synchronous merchant needs with an event stream for asynchronous network reality, we achieve both performance and correctness. This pattern, proven at companies like Stripe processing billions in payments, shows that the best distributed systems don't fight the nature of external dependencies — they embrace and design for them.

4) The system should be scalable to handle high transaction volume (10,000+ TPS)

We've come a long way! We have a payment processing system that meets just about all of our functional and non-functional requirements. Now we just need to discuss how we would handle scale. I like to save scale for my last deep dive because then I have a clear understanding of the full system I need to scale. It's likely that you already talked about scale (as we did a bit) during other parts of the interview, but we'll do our best to tie it all together here.

Servers

Let's start by acknowledging the basics. Each of our services is stateless and will scale horizontally with load balancers in front of them distributing the load. This is worth mentioning in the interview, but there is no need to draw it out; your interviewer knows what you mean, and this is largely taken for granted nowadays.

Kafka

Next, let's look deeper at our event log, Kafka. We are expecting ~10k tps. While **Kafka can support millions of messages per second at the cluster level**, it's important to note that each

individual partition typically handles around 5,000-10,000 messages per second under normal production conditions. To comfortably support our 10k TPS requirement, we'd need multiple partitions—likely 3-5—to ensure both throughput and fault tolerance.

When partitioning, we need to guarantee ordering for transactions per PaymentIntent. This suggests partitioning our Kafka topics by payment_intent_id, which ensures all events for a given PaymentIntent are processed in order while allowing parallel processing across different PaymentIntents. This partitioning strategy balances our throughput needs with our ordering requirements—all state transitions for a single PaymentIntent (created → authorized → captured) will be processed sequentially, while different PaymentIntents can be processed in parallel across partitions.

For redundancy, we'd configure Kafka with a replication factor of 3, meaning each partition has three copies distributed across different brokers. This provides fault tolerance if a broker fails. We'd also set up consumer groups for each of our services, allowing us to scale consumers horizontally while maintaining exactly-once processing semantics.

Learn more about how to scale Kafka in our Kafka deep dive here.

Database

Our database should be approximately the same order of magnitude as our event log with regards to the number of transactions per second. With 10k TPS for our event stream, we can expect around 10k writes per second to our database as well. This is right on the edge of what a well-optimized PostgreSQL instance can handle this load, especially when combined with read replicas and proper indexing. To better handle the scale, we can shard our database by merchant_id.

With regards to data growth, we can estimate that each row is ~500 bytes. This means we are storing `10,000 * 500 bytes = 5mb` of data a second, `5mb * 100,000 (rounded seconds in a day) = 500gb` of data a day, and `500gb * 365 = ~180tb` of data a year. This is significant storage growth that will require careful planning for data retention and archiving strategies.

For transactions older than a certain period (e.g., 3-6 months), we can move them to cold storage like Amazon S3 or Google Cloud Storage. This archived data would still be accessible for compliance and audit purposes but wouldn't impact our operational database

performance. We'd implement a scheduled job to identify, export, and purge old records according to our retention policy.

For read scaling, we can implement read replicas of our database. Most payment queries are read operations (checking payment status, generating reports), so having multiple read replicas will distribute this load. We can also implement a caching layer using Redis or Memcached for frequently accessed data like recent payment statuses.

Bonus Deep Dives

1) How can we expand the design to support Webhooks?

> As mentioned earlier, a robust webhook system is a substantial system design topic in its own right - one that could easily fill an entire interview (and often does!). In the context of this payment processor discussion, we'll outline the high-level approach to webhooks rather than diving into all implementation details.

While our polling-based status endpoint works well for basic scenarios, merchants often need real-time updates about payment status changes to trigger business processes like order fulfillment or access provisioning. Webhooks solve this by allowing our system to proactively notify merchants about events as they occur.

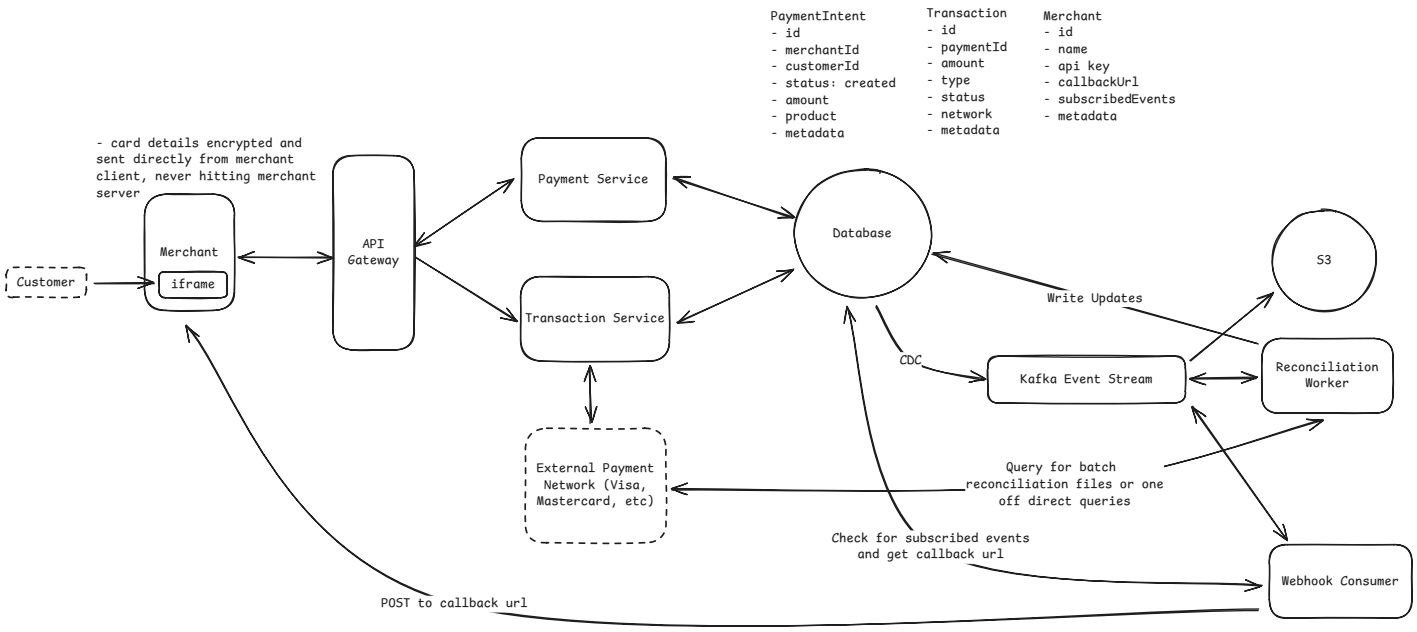
Merchants provide us with two additional bits of information:

1. Callback Url: This is the URL that we will `POST` updates to when we have them.
2. Subscribed Events: This is a list of events that they want to subscribe to. We will notify them, at the callback url, when any of these events occur.

> You hear real-time updates and you may think websockets or SSE! But no, these are often conflated concepts, but it's important to understand that webhooks represent server-to-server communication, not server-to-client (like websockets or SSE). The payment system's server sends notifications directly to the merchant's server via HTTP requests to a predefined endpoint. This is fundamentally different from client-facing real-time updates (like WebSockets or Server-Sent Events), which would deliver updates from a server to a

> browser or mobile app. Webhooks are designed for system-to-system communication and typically require the merchant to operate a publicly accessible endpoint to receive these notifications.

Here's how webhooks would work at a high level in our payment processing system:



Webhook System

1. **Database Changes**: Our Transaction and PaymentIntent services update the operational database as payments progress through their lifecycle (created → authorized → captured, etc.).
2. **CDC Events**: Change Data Capture automatically captures these database changes and publishes them to our Kafka event stream. These CDC events include payment status changes, transaction completions, and other state transitions.
3. **Webhook Service**: We introduce a new Webhook Service that consumes from the same Kafka event stream as our other specialized consumers ie, Reconciliation. When the service receives a CDC event, it checks if the associated merchant has configured a webhook endpoint for that event type. If configured, it prepares the webhook payload with relevant event details, signs the payload with a shared secret to enable verification, and attempts delivery to the merchant's endpoint.
4. **Delivery Management**: For each webhook, the delivery attempt is recorded with its status. If delivery fails, the Webhook Service implements a retry strategy with exponential

backoff (e.g., retry after 5s, 25s, 125s, etc., up to a reasonable maximum interval like 1 hour).

5. **Merchant Implementation**: On the merchant side, they would need to configure a publicly accessible HTTPS endpoint to receive webhooks. They must verify the signature of incoming webhooks using the shared secret to ensure authenticity. After verification, they would process the webhook payload and update their systems accordingly with the new information. Finally, they should return a 2xx HTTP status code to acknowledge receipt of the webhook, preventing unnecessary retries.

Simple example of a webhook payload:

```
{
  "id": "evt_1JklMnOpQrStUv",
  "type": "payment.succeeded",
  "created": 1633031234,
  "data": {
    "object": {
      "id": "pay_1AbCdEfGhIjKlM",
      "amountInCents": 2499,
      "currency": "usd",
      "status": "succeeded",
      "created": 1633031200
    }
  }
}
```

> In production systems, webhook delivery reliability is critical. Although we're not diving into all the details here, a complete webhook system would include features like idempotency keys, detailed delivery monitoring, webhook logs for debugging, and a dashboard for merchants to view and replay webhooks.

If this were a dedicated webhook system design interview, we would explore more intricate challenges such as exactly-once delivery semantics, handling webhook queue backlogs during outages, webhook payload versioning, and adaptive rate limiting to prevent overwhelming merchant systems. However, for our payment processor design, this high-level overview captures the essential functionality that would complement our core payment flows.

What is Expected at Each Level?

You may be thinking, "how much of that is actually required from me in an interview?" Let's break it down.

Mid-level

For a mid-level candidate discussing a payment system, I'd primarily focus on your ability to establish the core functionality. You should be able to design a straightforward payment flow that handles the basic requirements of initiating payment requests, processing cards, and returning status updates.

You should demonstrate understanding of the need for security in payment systems and identify that sensitive card data shouldn't touch merchant servers. However, I wouldn't expect you to deeply understand all the nuances of payment security or complex tokenization approaches. When discussing consistency, a reasonable solution like assuming timeouts mean failure (even if not ideal) would be acceptable as long as you can explain the approach clearly. I'd be looking more at your problem-solving process than expecting a perfect solution to complex payment reconciliation challenges.

I'd anticipate that you might need guidance on deeper technical challenges, and that's perfectly fine. The interviewer will likely direct the conversation through the latter portions of the discussion.

Senior

For a senior engineer, I'd expect you to move quickly through the basic functionality and drive the conversation toward the critical non-functional aspects of a payment system. You should proactively identify security and consistency as core challenges.

On the security front, you should propose a solution like the iframe isolation approach and understand its benefits in protecting sensitive card data. You should be able to articulate why this approach is superior to having merchants collect card data directly. For payment consistency, you should recognize that timeouts don't necessarily mean failure and propose a more sophisticated approach like idempotent transactions with pending resolution. You should

be able to identify potential race conditions and explain how your design prevents double-charging.

For scaling discussions, I'd expect you to speak with confidence about how you'd implement horizontal scaling for services and databases, though you might need some prompting to explore more advanced topics like Kafka partitioning strategies or event sourcing.

Staff+

As a staff+ candidate, I'd expect you to demonstrate deep expertise in payment systems design while also showing exceptional product and architectural thinking. Rather than rushing to complex solutions, you should first evaluate whether they're necessary. For instance, you might point out that for many payment scenarios, event sourcing with reconciliation is the right approach not just for technical reasons but because it aligns with how financial systems fundamentally work - acknowledging the inherently asynchronous nature of payment networks. For security, you should be able to explain multi-layered approaches like tokenization with client-side encryption, showing an understanding of the defense-in-depth strategy necessary for handling sensitive financial data.

You should proactively identify and address the most challenging edge cases in payment processing. For example, you might discuss how to handle payment network outages through fallback processing paths or how to design resilient reconciliation processes that guarantee eventual consistency with external payment networks.